discorso_tesi_20_minuti

Titolo: *Il problema del Neighborhood Search su grafi temporali*

Durata prevista: circa 20 minuti, con un ritmo naturale e brevi pause per le animazioni.

Le indicazioni tra parentesi quadre servono solo a orientarti durante la presentazione: non vanno lette. La parte tecnica della Variante 2 si trova dopo la slide di ringraziamento: è materiale di approfondimento per le domande, non fa parte del discorso.

Apertura e motivazione — da fare in 5 minuti circa

[Copertina]

Buonpomeriggio a tutti.

Oggi vi presenterò il lavoro svolto per la mia tesi magistrale, dal titolo «Il problema del Neighborhood Search su grafi temporali», sotto la supervisione del professor Luciano Gualà e del dottor Alessandro Straziota.

L'obiettivo di questa tesi è studiare come individuare in modo efficiente coppie di nodi con comportamenti relazionali simili all'interno di reti che evolvono nel tempo, combinando strutture dati per intervalli temporali e tecniche probabilistiche di ricerca per similarità.

[Contesto: reti sociali e articoli]

Pensiamo, per esempio, a una rete sociale. Se due utenti seguono molti degli stessi profili, potremmo considerarli simili, anche se non sono direttamente collegati. Un ragionamento analogo vale per una rete di citazioni: due articoli che fanno riferimento a molte delle stesse fonti possono avere argomenti vicini.

In entrambi gli esempi, quello che ci interessa è il **vicinato** di un nodo, cioè l'insieme dei nodi con cui è collegato. Il problema del *Neighborhood Search* nasce proprio da questa idea: dato un insieme di riferimento, vogliamo trovare un nodo il cui vicinato gli assomigli abbastanza.

[Definizione del problema statico, sempre visibile durante le animazioni]

Nel caso statico, abbiamo un grafo, una soglia di similarità e una funzione che misura quanto due insiemi si somigliano. La query è un insieme (S) di vertici; l'obiettivo è trovare un nodo (u) per il quale la similarità tra (S) e il suo vicinato raggiunga almeno la soglia richiesta.

Qui c'è una scelta importante: **che cosa intendiamo per «simili»?**

[Primo clic: compaiono le due coppie di insiemi]

Guardiamo le due situazioni rappresentate. A sinistra abbiamo due insiemi molto grandi, da mille elementi ciascuno, con una piccola parte in comune. A destra abbiamo invece due insiemi da dieci elementi, che si sovrappongono quasi completamente. I numeri all'interno dei cerchi ci aiutano a capire quanta parte di ciascun insieme sia condivisa con l'altro.

[Clic: compaiono le intersezioni]

Se usiamo soltanto il numero di elementi comuni, la prima coppia sembrerebbe migliore: venti elementi condivisi contro otto. È un risultato poco convincente, perché non considera la dimensione degli insiemi.

[Clic: compaiono i valori di Jaccard]

Per questo utilizziamo la **similarità di Jaccard**, che divide il numero di elementi comuni per il numero di elementi presenti nell'unione. Nell'esempio, la prima coppia ottiene circa 0,01; la seconda circa 0,67. Jaccard descrive quindi meglio la somiglianza relativa tra i vicinati ed è la misura adottata in questo lavoro.

[Ricerca esaustiva e LSH]

Il modo più immediato per rispondere a una query è confrontare (S) con il vicinato di ogni nodo (**ricerca esaustiva**). Funziona, ma su un grafo grande può richiedere moltissimi confronti.

Un'alternativa classica è utilizzare *MinHash* e *Locality Sensitive Hashing*, o LSH. In termini intuitivi, MinHash rappresenta un vicinato con una firma compatta; LSH usa queste firme per individuare i nodi che hanno maggiori possibilità di essere simili. A quel punto verifichiamo soltanto i candidati, invece di esaminare tutto il grafo. Il vantaggio è una ricerca più rapida, al prezzo di una possibile perdita di alcune risposte: il filtro, infatti, è probabilistico.

Dal grafo statico alle due varianti temporali — circa 3 minuti, con contributi

[Le reti cambiano nel tempo]

Finora abbiamo ragionato come se le relazioni fossero sempre presenti. In realtà, le interazioni avvengono in momenti precisi. Nell'esempio, il nodo (v) è collegato ad (a) e (b) al primo istante, ad (a) e (c) al secondo e a (b) e (c) al terzo. Se unissimo tutto in un solo grafo statico perderemmo l'informazione temporale. Per tale motivo si utilizza il modello del grafo temporale.

[Definizione di grafo temporale]

Un grafo temporale è un grafo in cui a ogni arco è associato l'insieme degli istanti in cui quell'arco è attivo. Possiamo quindi parlare di eventi, cioè interazioni tra due nodi in un preciso istante. Per ogni nodo definiamo inoltre (\Lambda(v)), l'insieme degli istanti in cui il nodo ha almeno un'interazione.

Il suo **vicinato temporale** su un intervallo (I) è dato dall'unione dei vicini incontrati in quell'intervallo. Qui, per esempio, nell'intervallo da 2 a 4 il nodo (v) incontra sia (a) sia (b), mentre da 5 a 6 incontra soltanto (a). Dunque, cambiando l'intervallo, cambia anche il vicinato che dobbiamo confrontare.

[Due varianti: primo overlay, poi Variante 1, poi Variante 2]

Il problema del NS su grafi temporali può essere espresso nel modo seguente: data una query costituita da un insieme di vertici, cerchiamo un nodo e una finestra temporale nei quali la similarità superi una certa soglia. La difficoltà aggiuntiva è che **non conosciamo in anticipo l'istante di inizio della finestra**: dobbiamo trovarlo durante la ricerca.

In questa tesi abbiamo definito ed analizzato due varianti. Nella **prima variante**, l'ampiezza (\mu) della finestra è fissata prima di costruire l'indice: tutte le query useranno quella stessa ampiezza. Nella **seconda variante**, invece, l'ampiezza viene scelta al momento della query. Questa differenza sembra piccola, ma cambia profondamente il modo in cui possiamo organizzare i dati.

[Sezione «Contributi» e le due slide riepilogative]

Analizziamo ora i contributi della tesi.

Per la prima variante il contributo si basa su 4 pilastri fondamentali: un'adattamento di LSH alle finestre temporali, accompagnato da un meccanismo di *pruning* e da un oracolo per recuperare rapidamente le firme MinHash. Verrà infine fornita un'analisi teorica dei bound, accompagnata da una campagna sperimentale. Per la seconda variante affrontiamo il problema da un altro punto di vista, sfruttando una rappresentazione geometrica degli intervalli.

Proponiamo quindi un filtro euristico basato sulla prima componente dello sketch MinHash: il filtro geometrico restituisce un **sovrainsieme** dei candidati selezionati dall'LSH tradizionale. Non abbiamo bound non banali per questo svorainsieme, ma grazie esperimenti vedremo che questo insieme rimane sorprendentemente contenuto.

Partiamo dalla prima variante, perché ci permette di vedere più chiaramente come entrano in gioco tutte queste idee.

Variante 1: come funziona la soluzione — circa 5/8 minuti compresi gli esperimenti(qua puoi perdere un po di tempo)

[Sezione «Variante 1» e slide LSH con bande]

Richiamiamo prima un dettaglio di LSH che ci servirà per adattarlo al tempo. Una firma MinHash è composta da (k) valori. LSH la divide in più blocchi, chiamati **bande**, e usa una tabella hash per ciascuna banda.

Nell'esempio, le firme di (A) e (B) sono diverse nel complesso, ma hanno la prima banda uguale. Questo è il punto fondamentale: non chiediamo che due firme coincidano interamente, perché altrimenti rischieremmo di escludere proprio gli insiemi che stiamo cercando.

[Clic: compaiono le tabelle, poi la collisione]

Per questo finiscono nello stesso contenitore della prima tabella. È sufficiente una collisione in almeno una banda perché i due insiemi siano considerati candidati. Il controllo della similarità viene dopo: essere candidati non significa ancora essere una risposta valida.

[Adattamento temporale: chiave con timestamp]

Come adattare questa idea a un grafo temporale? Potremmo costruire un indice LSH indipendente per ogni finestra di ampiezza (\mu). È una soluzione possibile, che useremo anche come termine di confronto, ma richiede di gestire molte strutture separate.

La nostra proposta usa invece **un unico gruppo di (b) tabelle hash condivise**. Per ogni nodo e per ogni finestra rilevante, recuperiamo la firma del suo vicinato temporale (grazie all'oracolo appositamente costruito, possiamo farlo in tempi molto veloci), la dividiamo in bande e la inseriamo nelle tabelle.

Il passaggio importante è la chiave: oltre al valore hash della banda, aggiungiamo **l'istante iniziale della finestra**. In questo modo due firme uguali possono generare una collisione utile soltanto se si riferiscono allo stesso intervallo temporale. Quindi condividiamo le tabelle, ma manteniamo distinti i diversi momenti della rete. È questo che ci consente di usare un indice unico per la prima variante.

[Pruning: timeline con eventi a 4 e 9]

L'idea del pruning nasce da una osservazione molto semplice. Se un nodo non ha alcuna interazione in una certa finestra, il suo vicinato su quella finestra è vuoto. Non ha quindi senso indicizzarlo come se contenesse informazioni utili.

Chiamiamo **(W(v))** l'insieme degli istanti iniziali delle finestre che contengono almeno un evento del nodo. Nell'esempio, consideriamo dodici istanti temporali, finestre di ampiezza due e un nodo attivo soltanto agli istanti quattro e nove. Le finestre possibili sono dieci. Di queste, sei incontrano almeno uno dei due eventi: le altre quattro vengono scartate.

Questa operazione si chiama *pruning*. Ogni istante di attività rende rilevanti al massimo (\mu+1) finestre; perciò possiamo limitare il numero di finestre da elaborare in funzione dell'attività del nodo. È importante, però, non attribuire al pruning un miglioramento che non ha: **nel caso peggiore tutte le finestre contengono eventi**, e allora non eliminiamo nulla. Il beneficio dipende da quanto le attività sono distribuite o concentrate nel tempo.

[Bound teorici]

I bound riportati qui riassumono proprio questa dipendenza. Il costo di costruzione e lo spazio dell'indice crescono con il numero di finestre effettivamente indicizzate; anche il tempo di query dipende da quante finestre attive dobbiamo considerare e da quanti candidati vengono poi verificati. Quindi la teoria chiarisce dove possiamo risparmiare lavoro, ma per capire quanto questo risparmio sia concreto dobbiamo guardare i dati reali.

Risultati sperimentali: efficienza e recall

[Il pruning nei dataset reali]

Abbiamo valutato il comportamento della soluzione su cinque dataset temporali reali. Il primo risultato è che l'efficacia del pruning varia molto da una rete all'altra.

Su **Enron**, un dataset di comunicazioni email, riusciamo a eliminare in media il **98,2%** delle finestre. Su **Facebook** il valore è ancora molto alto, **96,2%**. Su CollegeMsg e sul dataset degli scacchi il guadagno è più moderato. Infine, su **SFHH**, che raccoglie interazioni ravvicinate durante una conferenza, eliminiamo soltanto il **10,1%** delle finestre.

La ragione è legata alla distribuzione degli eventi. Quando un nodo è attivo in pochi periodi, possiamo escludere molti intervalli vuoti. Quando invece interagisce con continuità, quasi ogni finestra contiene almeno un evento, e il pruning ha poco spazio per intervenire.

[Approfondimento Enron]

Enron rende questa differenza particolarmente chiara. Nel grafico, il numero delle finestre rilevanti rimane molto più vicino al limite inferiore legato agli istanti di attività che al numero totale di finestre possibili. Passiamo quindi da 7900 finestre totali a circa 160 finestre in media.

[Confronto tra soluzione completa, LSH senza pruning e indici indipendenti]

Con il grafico successivo vediamo il confronto fra 3 implementazioni, sui tempi medi di query per il dataset Enron.

Con la soluzione che mantiene indici LSH indipendenti per finestra, indicata nel confronto come *Baseline*, il tempo è di **34,64 microsecondi**.

La **nostra soluzione completa**, con indice unificato e pruning, impiega mediamente **9,82 microsecondi** per query.

Se manteniamo l'indice unificato ma togliamo il pruning, il tempo sale a circa **91,88 microsecondi**.

Questo risultato è interessante perché mostra che il miglioramento non dipende da un singolo accorgimento. L'indice condiviso evita di gestire una grande quantità di strutture separate; il pruning evita di interrogare finestre che non servono. Insieme, queste due scelte riducono il lavoro effettivo della ricerca.

[Recall]

Naturalmente, una ricerca veloce ha poco valore se perde troppe risposte. Per questo abbiamo valutato anche la **recall** della nostra soluzione, rispetto alla ground truth (insieme delle soluzioni ammissibili).

Con i parametri (k=128), (b=8), (r=16) e soglia (0,35), la recall raggiunge **98,07% su Enron** e **93,32% su Facebook**, mentre su **CollegeMsg** scende al **67,31%**. Dunque il filtro può introdurre falsi negativi, e la loro frequenza dipende dal dataset e da come sono distribuite le similarità tra i vicinati. È il principale compromesso di questo approccio: otteniamo query molto rapide, ma la completezza delle risposte non è garantita. Bisogna osservare che , cambiando i parametri, e sopratutto aumentando K e b si potrebbero ottenere risultati ancora migliori.

Variante 2 e conclusioni — circa 2 minuti e 30 secondi

[Sezione «Variante 2»]

Passiamo ora alla **seconda variante**, nella quale l'ampiezza della finestra non è fissata in anticipo, ma viene specificata insieme alla query. È una situazione più flessibile, ma anche più difficile da indicizzare: non possiamo preparare in anticipo una struttura distinta per ogni possibile ampiezza senza far crescere enormemente lo spazio richiesto.

Abbiamo quindi studiato un filtro euristico basato su una rappresentazione geometrica degli intervalli temporali. **Per questioni di tempo non entrerò nei dettagli tecnici di questa costruzione né nei suoi bound**. Ho lasciato le relative slide di approfondimento in fondo alla presentazione, dopo le conclusioni e i ringraziamenti, e sarò naturalmente disponibile a discuterle durante le domande della commissione.

In questa parte vorrei invece mostrarvi soltanto un risultato molto concreto: **quanto è selettivo il filtro**, cioè quanti nodi ci lascia effettivamente da controllare.

[Selettività sperimentale: CollegeMsg ed Enron]

Su **CollegeMsg**, che contiene **1.899 nodi**, il filtro individua in media tra circa **33 e 64 candidati**, a seconda dell'ampiezza della finestra. Anche nel caso con la media più alta, parliamo soltanto del **3,39%** dei nodi.

Su **Enron**, la differenza è ancora più evidente: su oltre **87 mila nodi**, il filtro restituisce mediamente appena **219–230 candidati**, cioè circa lo **0,26%** dell'intero grafo. Quindi, pur lavorando su una rete molto più grande, possiamo concentrare la verifica successiva su una frazione davvero piccola dei nodi.

[Conclusioni]

In conclusione, i risultati principali sono due.

Per la **Variante 1**, l'indice LSH unificato, l'oracolo e il pruning permettono di ridurre il lavoro sulle finestre temporali: su **Enron eliminiamo il 98,2% delle finestre**, con tempi di query molto contenuti.

Per la **Variante 2**, il risultato più significativo è la selettività del filtro: su un dataset di oltre 87 mila nodi, i candidati medi sono circa **230**, appena lo **0,26%** del totale.

I **prossimi passi** sono completare la fase di verifica della seconda variante, misurarne tempi e recall, esplorare altre definizioni del problema e studiare configurazioni LSH capaci di migliorare il compromesso tra efficienza e qualità delle risposte.

[Ringraziamenti]

Vi ringrazio per l'attenzione. Sono a disposizione per eventuali domande e, se di interesse, per approfondire la costruzione geometrica e i bound della seconda variante, riportati nelle slide successive.